

Supplementary Materials: Structured Schemas for LLM-Modeler Collaboration in Quantitative Systems Pharmacology Model Calibration

Joel Eliason and Aleksander S. Popel

Contents

1	Complete SubmodelTarget Example	3
1.1	Overview	3
1.2	Complete YAML Specification	3
1.3	Schema Structure Summary	7
2	Supported Model Types	8
3	Validation Checks	10
3.1	Summary of Validators	10
3.2	DOI Resolution Validator	10
3.3	Value-in-Snippet Validator	11
3.4	Unit Validation	12
3.5	Reference Consistency Validators	13
3.6	Code Execution Validation	14
3.7	Hardcoded Constant Detection	14
3.8	Scale Consistency Validation	14
3.9	Control Character Validation	14
3.10	CalibrationTarget-Specific Validation	14
4	Generated Julia Code Structure	14
5	Pydantic Model Definitions	15
5.1	Top-Level Model	15
5.2	Input Model	16
5.3	Model Type Discriminated Union	17
5.4	Prior Specification	18
5.5	Loading and Validating YAML Files	19
5.6	CalibrationTarget Model	20
5.7	Observable	20
5.8	CalibrationTargetEstimates	21
5.9	Scenario and Intervention	22

33	6 SubmodelTarget Schema Details	23
34	6.1 Input Fields	23
35	6.2 Parameter and Prior Specification	24
36	6.3 Forward Model Specification	24
37	6.4 Error Model Specification	25
38	7 Complete CalibrationTarget Example	26
39	7.1 Overview	26
40	7.2 Observable	26
41	7.3 Empirical Data	27
42	7.4 Source and Context	29
43	8 SubmodelTarget Source Characteristics	30
44	8.1 Source Quality	30
45	8.2 Species Translation	30
46	8.3 Indication Match	31
47	9 CalibrationTarget Detailed Metrics	31
48	9.1 Observable Categories	31
49	9.2 Observable Complexity	31
50	9.3 Extraction Model Attribution	32
51	9.4 Primary Data Sources	32
52	10 Inference Results	32
53	10.1 Convergence Diagnostics	33
54	10.2 Posterior Distributions	33
55	11 Model-Aware Prompt Construction Details	36
56	12 Schema Implementation Details	36
57	13 Source Relevance Assessment Details	37
58	14 Collaboration Mode Details	37
59	15 Detailed Comparison to Existing Approaches	38

1 Complete SubmodelTarget Example

This section presents a complete SubmodelTarget for activated pancreatic stellate cell (aPSC) proliferation, demonstrating all schema components.

1.1 Overview

This target constrains the proliferation rate parameter `k_apsc_prolif` using data from Schneider et al. (2001), who measured PSC proliferation in response to PDGF stimulation via BrdU incorporation assay.

1.2 Complete YAML Specification

```
# Schema: SubmodelTarget
# Target: Activated PSC proliferation rate

target_id: psc_proliferation_PDAC_deriv001

# =====
# INPUTS: Extracted values with full provenance
# =====
inputs:
  - name: pdgf_fold_mean
    value: 4.37
    units: dimensionless
    input_type: direct_measurement
    role: target
    source_ref: Schneider2001_PSC_PDGF
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49- and 2.96 +/-
                   0.39-fold of control)."
```

```
  - name: pdgf_fold_sd
    value: 0.49
    units: dimensionless
    input_type: direct_measurement
    role: auxiliary
    source_ref: Schneider2001_PSC_PDGF
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49- and 2.96 +/-
                   0.39-fold of control)."
```

```
# =====
# CALIBRATION: Everything needed for inference
# =====
calibration:
  parameters:
    - name: k_apsc_prolif
```

```

103     units: 1/day
104     prior:
105         distribution: lognormal
106         mu: 0.0      # log(1.0) - expect ~1/day for 4x fold-change in 1 day
107         sigma: 1.0
108         rationale: |
109             Wide log-normal prior centered at 1/day. A 4-fold increase in
110             1 day implies  $k = \ln(4) = 1.4/\text{day}$ , so prior median of 1/day
111             is reasonable.
112
113 forward_model:
114     type: exponential_growth
115     rate_constant: k_apsc_prolif
116     data_rationale: |
117         BrdU incorporation assay measures proliferation as fold-change
118         over ~24 hours under constant PDGF stimulus. With a single high
119         PDGF dose and short duration, exponential growth ( $dN/dt = k \cdot N$ )
120         fits the data structure.
121     submodel_rationale: |
122         The full QSPiO_PDAC model has aPSC proliferation via:
123          $k_{\text{apsc\_prolif}} * \text{aPSC} * (\text{PDGF} / (\text{PDGF}_{50\_prolif} + \text{PDGF}))$ .
124         At 50 ng/mL PDGF  $\gg$  PDGF50_prolif, the saturation term  $\sim 1$ ,
125         reducing to exponential growth with rate  $k_{\text{apsc\_prolif}}$ .
126     independent_variable:
127         name: time
128         units: day
129         span: [0.0, 1.0]
130         rationale: |
131             Proliferation assay duration assumed to be ~24 hours based on
132             typical cytokine incubation protocols for rat PSCs.
133     state_variables:
134         - name: aPSC_rel
135           units: dimensionless
136           initial_condition:
137               value: 1.0
138               rationale: |
139                   PSC proliferation data are reported as fold of control. We
140                   normalize the initial aPSC population to 1.0 (100% control)
141                   to match this convention.
142
143 error_model:
144     - name: proliferation_fold_change
145       observable:
146           type: identity
147           state_variables: [aPSC_rel]
148           rationale: |
149               The state variable aPSC_rel is the fold-change in aPSC number
150               relative to baseline (control = 1.0), which directly matches

```

```

151         the reported proliferation fold-change.
152     units: dimensionless
153     uses_inputs:
154         - pdgf_fold_mean
155         - pdgf_fold_sd
156     evaluation_points: [1.0]
157     sample_size: 3
158     sample_size_rationale: |
159         Sample size not explicitly reported; assumed n=3 independent
160         experiments, typical for in vitro PSC BrdU proliferation assays.
161     distribution_code: |
162         def derive_distribution(inputs, ureg):
163             import numpy as np
164
165             rng = np.random.default_rng(42)
166
167             mean = inputs['pdgf_fold_mean'].magnitude
168             sd = inputs['pdgf_fold_sd'].magnitude
169             n = 10000
170
171             # Use lognormal for positive fold-changes
172             mu_log = np.log(mean**2 / np.sqrt(mean**2 + sd**2))
173             sigma_log = np.sqrt(np.log(1 + sd**2 / mean**2))
174
175             samples = rng.lognormal(mu_log, sigma_log, n)
176
177             median_val = np.median(samples)
178             ci_lower = np.percentile(samples, 2.5)
179             ci_upper = np.percentile(samples, 97.5)
180
181             return {
182                 'median': [median_val],
183                 'ci95': [[ci_lower, ci_upper]],
184                 'units': 'dimensionless'
185             }
186     likelihood:
187         distribution: lognormal
188         rationale: |
189             Fold-changes are positive and multiplicative, making
190             lognormal appropriate.
191
192     identifiability_notes: |
193         With a single timepoint (~24h) and normalized initial condition
194         (aPSC_rel(0)=1), only k_apsc_prolif is estimated. If substantial
195         PSC death occurred over the assay interval, the inferred rate
196         would represent net growth rather than pure proliferation.
197
198     # =====

```

```

199 # EXPERIMENTAL CONTEXT
200 # =====
201 experimental_context:
202     species: rat
203     system: in_vitro_primary_cells
204     indication: PDAC
205     cell_types:
206         - name: pancreatic stellate cell
207           phenotype: activated (alpha-SMA positive)
208           isolation_method: Isolated from normal Wistar rat pancreas,
209                           activated by culture on plastic
210     culture_conditions:
211         culture_type: 2d_monolayer
212
213 # =====
214 # STUDY INTERPRETATION
215 # =====
216 study_interpretation: |
217     Schneider et al. (2001) measured proliferation of cultured rat
218     pancreatic stellate cells (PSCs) in response to growth factors using
219     BrdU incorporation. Addition of 50 ng/mL PDGF increased PSC
220     proliferation to 4.37 +/- 0.49-fold of control, indicating that PDGF
221     acts as a strong mitogen for activated PSCs. We interpret the
222     50 ng/mL PDGF condition as approximating a near-maximal PDGF stimulus
223     and use the reported fold-change over ~24 hours as a quantitative
224     constraint on the maximal PDGF-driven proliferation rate parameter
225     k_apsc_prolif.
226
227 key_assumptions:
228     - |
229         50 ng/mL PDGF corresponds to a near-saturating concentration for
230         PSC proliferation, so  $PDGF/(PDGF_{50\_prolif} + PDGF) \sim 1$ .
231     - |
232         The proliferation assay duration is approximately 24 hours,
233         consistent with typical cytokine incubation protocols.
234     - |
235         BrdU incorporation fold-change is proportional to fold-change in
236         PSC cell number (negligible death over 24 hours).
237     - |
238         Rat activated PSC proliferation kinetics in vitro provide an upper
239         bound for human tumor aPSC proliferation in vivo.
240
241 key_study_limitations:
242     - |
243         Proliferation was measured via BrdU incorporation rather than
244         direct cell counts.
245     - |
246         Single PDGF concentration and single assay interval constrain

```

```

247     only the maximal rate.
248 - |
249     Data from rat PSCs in vitro, not human PDAC-associated stellate cells.
250 - |
251     Background serum proliferative signals not explicitly modeled.
252
253 # =====
254 # DATA SOURCES
255 # =====
256 primary_data_source:
257     doi: "10.1152/ajpcell.2001.281.2.C532"
258     title: "Identification of mediators stimulating proliferation and
259           matrix synthesis of rat pancreatic stellate cells"
260     authors: [Schneider, Schmid, Liptay, Schmid, Pohl]
261     year: 2001
262     source_tag: Schneider2001_PSC_PDGF
263
264 secondary_data_sources:
265 - doi: "10.1136/gut.44.4.534"
266   title: "Pancreatic stellate cells are activated by proinflammatory
267         cytokines: implications for pancreatic fibrogenesis"
268   authors: [Apte, Haber, Darby, Rodgers, McCaughan, Korsten, Pirola, Wilson]
269   year: 1999
270   source_tag: Apte1999_PSC_Cytokines
271   contribution: |
272     Provides context on PSC activation and cytokine response protocols.

```

273 1.3 Schema Structure Summary

274 The SubmodelTarget schema has the following top-level structure:

- 275 • **target_id**: Unique identifier for the target
- 276 • **inputs**: List of values extracted from literature with provenance
- 277 • **calibration**: Everything needed for inference code generation
 - 278 - **parameters**: Parameters to estimate with priors
 - 279 - **forward_model**: Forward model specification (built-in type or custom). For ODE mod-
 280 els, contains nested **state_variables** (state variables with initial conditions) and **independent_variabl**
 281 (e.g. time)
 - 282 - **error_model**: Observable specifications mapping model predictions to data with likeli-
 283 hoods
 - 284 - **identifiability_notes**: Discussion of parameter identifiability
- 285 • **experimental_context**: Species, system, cell types, culture conditions
- 286 • **study_interpretation**: Scientific interpretation of the data
- 287 • **key_assumptions**: Assumptions made in using this data

- 288 • **key_study_limitations:** Known limitations of the source study
- 289 • **primary_data_source:** Primary literature reference with DOI
- 290 • **secondary_data_sources:** Additional references

291 **2 Supported Model Types**

292 The SubmodelTarget schema supports 15 built-in forward model types, organized into four cate-
293 gories.

Table 1: Built-in forward model types organized by category.

Category	Model Type	Formulation	Key Fields
ODE templates	exponential_growth	$dy/dt = k \cdot y$	rate_constant
	first_order_decay	$dy/dt = -k \cdot y$	rate_constant
	logistic	$dy/dt = k \cdot y(1 - y/K)$	rate_constant, carrying_capacity
	saturation	$dy/dt = k(1 - y)$	rate_constant
	two_state	$dA/dt = -kA, dB/dt = kA$	forward_rate
	michaelis_menten	$dy/dt = -V_{\max}y/(K_m + y)$	vmax, km
Structured steady-state	steady_state_density	rate = density $\times$ loss	target_rate, observed_density, loss_rate
	steady_state_fraction	rate = $f \times \rho_{\text{parent}} \times \text{loss}$	target_rate, observed_fraction, parent_density
	steady_state_concentration	sec = $C \times k_{\text{cl}} \times V$	target_rate, observed_concentration, clearance_rate
	steady_state_ratio	$r = k_1/k_2$	target_rate, observed_ratio, opposing_rate
	steady_state_proliferation	$f = k_p/(k_p + k_d)$	target_rate, observed_fraction, death_rate
Accumulation	batch_accumulation	$C = (\text{sec} \times n \times t)/V$	target_rate, cell_count, incubation_time
Generic/ fallback	algebraic	(closed-form formula)	code, code_julia
	direct_fit	(curve fitting)	curve
	custom	(user-provided ODE)	code, code_julia

Each structured steady-state and accumulation model declares its fields as typed roles: a field can reference a calibration parameter (to be estimated), an extracted input (from the inputs layer), a curated reference value from a shared database, or a numeric literal. This role-based design allows the same template to serve different estimation scenarios depending on which field is treated as the unknown.

3 Validation Checks

The validation framework includes ten automated checks implemented as Pydantic `@model_validator` methods. Each validator runs after field parsing and raises a `ValidationError` with detailed messages when constraints are violated.

3.1 Summary of Validators

1. **Input reference validation:** All `uses_inputs` references match defined input names
2. **Source reference validation:** All `source_ref` values match defined source tags
3. **Parameter reference validation:** Model parameter roles reference defined parameters
4. **State variable reference validation:** Observable `state_variables` match defined state variables
5. **DOI resolution:** All DOIs resolve via CrossRef API
6. **Value-in-snippet validation:** Extracted values appear verbatim in quoted snippets
7. **Unit validation:** All unit strings parse with Pint
8. **Code syntax validation:** Custom code passes Python AST parsing
9. **Span ordering validation:** Time spans have `start < end`
10. **ODE model requirements:** ODE models have required state variables and spans

3.2 DOI Resolution Validator

The DOI validator queries the CrossRef API to verify that every cited DOI resolves to a valid publication. This catches fabricated citations before they enter the calibration pipeline. The validator also performs fuzzy matching on title and year to detect metadata inconsistencies.

```
def resolve_doi(doi: str) -> Optional[dict]:
    """Resolve DOI and get metadata from CrossRef."""
    doi_clean = doi.replace("https://doi.org/", "").strip()

    url = f"https://doi.org/{doi_clean}"
    headers = {"Accept": "application/vnd.citationstyles.csl+json"}
    response = requests.get(url, headers=headers, timeout=10)

    if response.status_code != 200:
        return None # DOI does not resolve
```

329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355

```
metadata = response.json()
return {
    "title": metadata.get("title", ""),
    "first_author": metadata.get("author", [{}])[0].get("family", ""),
    "year": metadata.get("issued", {}).get("date-parts", [[]])[0][0]
}

@model_validator(mode="after")
def validate_doi_resolution_and_metadata(self) -> "SubmodelTarget":
    """Validate that DOIs resolve and metadata matches."""
    errors = []

    metadata = resolve_doi(self.primary_data_source.doi)
    if metadata is None:
        errors.append(f"DOI '{self.primary_data_source.doi}' failed to resolve")
    else:
        # Check title match (fuzzy)
        if not fuzzy_match(self.primary_data_source.title, metadata["title"]):
            errors.append(f"Title mismatch with CrossRef")
        # Check year match
        if self.primary_data_source.year != metadata["year"]:
            errors.append(f"Year mismatch: {self.primary_data_source.year} vs {metadata['year']}")

    if errors:
        raise ValueError("DOI validation errors:\n - " + "\n - ".join(errors))
    return self
```

356 **Example failure:** If an LLM fabricates a citation with DOI 10.1234/fake.2023.12345, the
357 CrossRef query returns a 404 error and validation fails with:

358 `ValidationError: DOI '10.1234/fake.2023.12345' failed to resolve.`
359 `Verify at https://doi.org/10.1234/fake.2023.12345`

360 3.3 Value-in-Snippet Validator

361 The value-in-snippet validator checks that each extracted numeric value appears in its associated
362 source text. This catches hallucinations where an LLM extracts a plausible but incorrect value. The
363 validator handles multiple numeric formats including scientific notation, percentages, and Unicode
364 superscripts.

```
def check_value_in_text(text: str, value: float) -> bool:
    """Check if numeric value appears in text."""
    text_norm = text.lower().replace(",", "")
    patterns = []

    # Direct value and common formats
    patterns.append(str(value))
    patterns.append(f"{value:.1f}")
```

```

373     patterns.append(f"{value:.2f}")
374
375     # Scientific notation (e.g., "1.5e-3", "1.5E-03")
376     if abs(value) < 0.01 or abs(value) > 1000:
377         patterns.append(f"{value:e}")
378         patterns.append(f"{value:.2e}")
379
380     # Percentage format (if value is between 0 and 1)
381     if 0 < value < 1:
382         pct = value * 100
383         patterns.extend([f"{pct}%", f"{pct:.0f}%", f"{pct:.1f}%"])
384
385     return any(p.lower() in text_norm for p in patterns)
386
387 @model_validator(mode="after")
388 def validate_input_values_in_snippets(self) -> "SubmodelTarget":
389     """Validate that extracted values appear in their value_snippet."""
390     errors = []
391     for inp in self.inputs:
392         if not inp.value_snippet:
393             continue
394         # Skip types that may not have explicit values in text
395         if inp.input_type in (InputType.INFERRED_ESTIMATE, InputType.ASSUMED_VALUE):
396             continue
397
398         if not check_value_in_text(inp.value_snippet, inp.value):
399             errors.append(
400                 f"Input '{inp.name}': value {inp.value} not found in snippet "
401                 f"'{inp.value_snippet[:60]}...'")
402         )
403
404     if errors:
405         raise ValueError("Value-snippet mismatches (possible hallucination):\n - "
406                          + "\n - ".join(errors))
407     return self

```

408 **Example failure:** If an input claims value: 4.37 but the snippet reads “Cell proliferation
409 was 3.21-fold of control”, validation fails with:

```

410 ValidationError: Value-snippet mismatches (possible hallucination):
411 - Input 'pdgf_fold_mean': value 4.37 not found in snippet
412 'Cell proliferation was 3.21-fold of control...'

```

413 This validator is particularly effective because LLMs that hallucinate values rarely fabricate
414 consistent snippets, so the mismatch provides a reliable detection signal.

415 3.4 Unit Validation

416 The unit validator parses all unit strings using Pint, a Python library for physical quantities.
417 Malformed or unrecognized units trigger validation failure.

```

418 @model_validator(mode="after")
419 def validate_units_are_valid_pint(self) -> "SubmodelTarget":
420     """Validate that all unit strings are valid Pint units."""
421     from pint import UnitRegistry
422     ureg = UnitRegistry()
423     errors = []
424
425     def check_unit(unit_str: str, location: str):
426         try:
427             ureg(unit_str)
428         except Exception as e:
429             errors.append(f"{location}: '{unit_str}' is not valid ({e})")
430
431     for inp in self.inputs:
432         check_unit(inp.units, f"Input '{inp.name}'")
433     for param in self.calibration.parameters:
434         check_unit(param.units, f"Parameter '{param.name}'")
435
436     if errors:
437         raise ValueError("Invalid Pint units:\n - " + "\n - ".join(errors))
438     return self

```

439 **Example failure:** If a parameter specifies units: `cells/mL/day` (invalid compound unit),
440 validation fails with:

```

441 ValidationError: Invalid Pint units:
442 - Parameter 'k_recruit': 'cells/mL/day' is not valid (...)

```

443 3.5 Reference Consistency Validators

444 Four validators check that all cross-references within the schema are consistent:

- 445 • **Input references:** Every `uses_inputs` entry and `initial_condition.input_ref` must
446 match a defined input name.
- 447 • **Source references:** Every `source_ref` must match `primary_data_source.source_tag` or a
448 `secondary_data_sources[].source_tag`.
- 449 • **Parameter references:** When a model field like `rate_constant` contains a string (not an
450 `InputRef`), it must match a parameter name in `calibration.parameters`.
- 451 • **State variable references:** Every state variable referenced in `observable.state_variables`
452 must exist in `calibration.state_variables`.

453 These validators catch internal inconsistencies where an LLM generates references to entities
454 that don't exist elsewhere in the document.

3.6 Code Execution Validation

Code validation goes beyond syntax checking. The validator parses the code structure, then executes it with mock data to verify correct behavior. For `observation_code`, execution must return the required dictionary keys (`value`, `sd`, `sd_uncertain`) with appropriate types. For observable code, execution must return arrays with correct units and dimensions. This catches logic errors and signature mismatches before inference.

3.7 Hardcoded Constant Detection

Hardcoded constant detection scans code blocks for numeric literals that should be documented inputs. The parser identifies constants (e.g., 1440 for a minutes-to-days conversion) that are not in the allowed set (small integers 0–5, common fractions, statistical percentiles). Each flagged constant must either be moved to an input with provenance or documented as an `observable_constant` with a `biological_basis` explanation.

3.8 Scale Consistency Validation

Scale consistency validation executes observable code with mock species data spanning a 1000-fold range. If the output is constant despite varying inputs, or if the output magnitude suggests a ratio/score mismatch (e.g., values near 100 when 0–1 is expected), validation warns of potential unit or scaling errors.

3.9 Control Character Validation

Control character validation scans all string fields for characters that corrupt YAML parsing (ASCII control codes, non-breaking spaces, and similar). PDF copy-paste often introduces invisible characters that cause parsing failures; this validator catches them before they propagate.

3.10 CalibrationTarget-Specific Validation

CalibrationTarget adds validators for full-model observables. Observable code is executed against mock species data to verify correct function signatures, output dimensionality, and unit consistency. A denominator audit requires that density or fraction observables explicitly document what tissue area or cell population serves as the denominator, both experimentally and in the model, catching a common source of order-of-magnitude errors. Species existence checks verify that every species referenced in the observable code exists in the model structure. Distribution code is executed with the declared inputs to verify that computed summary statistics (median and CI95) match the values reported in the empirical data block within tolerance (1% for median, 10% for CI bounds), catching transcription errors and code bugs before they propagate to calibration.

4 Generated Julia Code Structure

The translator generates Julia scripts with the following structure:

```
using DifferentialEquations
using Turing
using Distributions
```

```

492 # Observed data (from inputs layer)
493 const obs_proliferation_median = 4.37
494 const obs_proliferation_sigma = 0.25 # derived from CI95
495
496 # ODE function
497 function ode_proliferation!(du, u, p, t)
498     k = p[1]
499     du[1] = k * u[1] # exponential growth
500 end
501
502 # Simulate function
503 function simulate_proliferation(k; t_span=(0.0, 1.0), u0=[1.0])
504     prob = ODEProblem(ode_proliferation!, u0, t_span, [k])
505     sol = solve(prob, Tsit5())
506     return sol[end][1] # final value
507 end
508
509 # Turing model
510 @model function proliferation_model()
511     # Prior
512     k_apsc_prolif ~ LogNormal(0.0, 1.0)
513
514     # Simulate
515     pred = simulate_proliferation(k_apsc_prolif)
516
517     # Likelihood
518     obs_proliferation_median ~ LogNormal(log(pred), obs_proliferation_sigma)
519 end
520
521 # Run inference
522 model = proliferation_model()
523 chain = sample(model, NUTS(), MCMCThreads(), 1000, 4)

```

524 For joint inference over multiple targets, the translator identifies shared parameters by name
525 and generates a single model with one prior per unique parameter.

526 5 Pydantic Model Definitions

527 This section provides the key Pydantic model definitions that implement both schemas. The full
528 implementations are available in the repository at `src/qsp_llm_workflows/core/calibration/`.

529 5.1 Top-Level Model

530 The `SubmodelTarget` class is the entry point for validation. When a YAML file is loaded, Pydantic
531 recursively validates all nested models.

```

532 class SubmodelTarget(BaseModel):
533     """

```

```

534     Submodel-based calibration target for QSP parameter inference.
535
536     Separates:
537     - 'inputs': What was extracted from papers (with full provenance)
538     - 'calibration': How to use those inputs for inference
539     """
540     target_id: str = Field(description="Unique identifier for this target")
541
542     # Extracted data with provenance
543     inputs: List[Input] = Field(
544         description="Values extracted from literature with full provenance"
545     )
546
547     # Calibration specification
548     calibration: Calibration = Field(
549         description="Everything needed for inference code generation"
550     )
551
552     # Experimental context
553     experimental_context: ExperimentalContext = Field(
554         description="Experimental context for the target"
555     )
556
557     # Study interpretation
558     study_interpretation: str = Field(
559         description="Scientific interpretation of how the study data informs the model"
560     )
561     key_assumptions: List[str] = Field(
562         description="Key assumptions made in using this data"
563     )
564     key_study_limitations: Optional[List[str]] = Field(
565         default=None, description="Known limitations of the study"
566     )
567
568     # Data sources
569     primary_data_source: PrimaryDataSource = Field(
570         description="Primary literature source"
571     )
572     secondary_data_sources: Optional[List[SecondaryDataSource]] = Field(
573         default=None, description="Additional literature sources"
574     )
575
576     # Validators (10 total) defined via Pydantic decorators
577     # See Section S3 for implementation details

```

5.2 Input Model

Each input captures a single value extracted from literature with full provenance:


```

580 class InputType(str, Enum):
581     """Type of input extracted from literature."""
582     DIRECT_MEASUREMENT = "direct_measurement" # Value reported directly
583     PROXY_MEASUREMENT = "proxy_measurement" # Requires conversion
584     EXPERIMENTAL_CONDITION = "experimental_condition" # Protocol choice
585     INFERRED_ESTIMATE = "inferred_estimate" # Interpreted from qualitative text
586     ASSUMED_VALUE = "assumed_value" # Domain knowledge, not in source
587
588 class InputRole(str, Enum):
589     """Role of input in calibration."""
590     INITIAL_CONDITION = "initial_condition" # IC for ODE integration
591     TARGET = "target" # Calibration target (likelihood term)
592     FIXED_PARAMETER = "fixed_parameter" # Fixed value in model
593     AUXILIARY = "auxiliary" # Supporting data
594
595 class Input(BaseModel):
596     """A value extracted from literature with full provenance."""
597     name: str = Field(description="Unique identifier for this input")
598     value: float = Field(description="Extracted numeric value")
599     units: str = Field(description="Units of the value")
600     uncertainty: Optional[Uncertainty] = Field(default=None)
601     n: Optional[int] = Field(default=None, description="Sample size")
602     input_type: InputType
603     role: Optional[InputRole] = Field(default=None)
604     source_ref: str = Field(description="Reference to source_tag in data sources")
605     source_location: str = Field(description="Location within the source")
606     value_snippet: Optional[str] = Field(
607         default=None,
608         description="Exact text from paper containing the value (for validation)"
609     )

```

610 5.3 Model Type Discriminated Union

611 The schema uses Pydantic's discriminated union pattern to support multiple model types, each
612 with type-specific required fields:

```

613 class BaseForwardModelSpec(BaseModel):
614     """Base class for all forward model specifications."""
615     data_rationale: str = Field(
616         description="Why this model type fits the experimental data"
617     )
618     submodel_rationale: str = Field(
619         description="Why this is a valid submodel of the full QSP model"
620     )
621
622 class ExponentialGrowthModel(BaseForwardModelSpec):
623     """Exponential growth:  $dy/dt = k * y$ """
624     type: Literal["exponential_growth"] = "exponential_growth"

```

```

625     rate_constant: ParameterRole = Field(
626         description="Rate constant parameter name or input_ref"
627     )
628
629 class TwoStateModel(BaseForwardModelSpec):
630     """Two-state transition: A -> B with first-order kinetics."""
631     type: Literal["two_state"] = "two_state"
632     forward_rate: ParameterRole = Field(
633         description="Forward transition rate constant"
634     )
635
636 class CustomODEModel(BaseForwardModelSpec):
637     """Custom ODE with user-provided code."""
638     type: Literal["custom_ode"] = "custom_ode"
639     code: str = Field(description="Python ODE function")
640     code_julia: str = Field(description="Julia ODE function for inference")
641
642 # Discriminated union selects model class based on 'type' field
643 ForwardModel = Annotated[
644     Union[
645         FirstOrderDecayModel,
646         ExponentialGrowthModel,
647         LogisticModel,
648         MichaelisMentenModel,
649         TwoStateModel,
650         SaturationModel,
651         SteadyStateDensityModel,
652         SteadyStateFractionModel,
653         SteadyStateConcentrationModel,
654         SteadyStateRatioModel,
655         SteadyStateProliferationIndexModel,
656         BatchAccumulationModel,
657         AlgebraicModel,
658         DirectFitModel,
659         CustomODEModel,
660     ],
661     Field(discriminator="type"),
662 ]

```

The `discriminator="type"` annotation tells Pydantic to inspect the `type` field in the YAML to determine which model class to instantiate. This ensures that an `exponential_growth` model is validated against `ExponentialGrowthModel` (which requires `rate_constant`) while a `two_state` model is validated against `TwoStateModel` (which requires `forward_rate`).

5.4 Prior Specification

Priors are specified with distribution type and parameters:

```

669 class PriorDistribution(str, Enum):

```

```

670     """Supported prior distribution types."""
671     LOGNORMAL = "lognormal"      # For positive parameters (rates, densities)
672     NORMAL = "normal"           # For unconstrained parameters
673     UNIFORM = "uniform"         # For bounded parameters
674     HALF_NORMAL = "half_normal" # For positive parameters with mode at 0
675
676 class Prior(BaseModel):
677     """Prior distribution specification for Bayesian inference."""
678     distribution: PriorDistribution
679     mu: Optional[float] = Field(default=None, description="Location parameter")
680     sigma: Optional[float] = Field(default=None, description="Scale parameter")
681     lower: Optional[float] = Field(default=None, description="Lower bound")
682     upper: Optional[float] = Field(default=None, description="Upper bound")
683     rationale: Optional[str] = Field(default=None)
684
685     @model_validator(mode="after")
686     def validate_prior_params(self) -> "Prior":
687         """Validate that required parameters are provided for each distribution."""
688         if self.distribution == PriorDistribution.LOGNORMAL:
689             if self.mu is None or self.sigma is None:
690                 raise ValueError("lognormal prior requires mu and sigma")
691             elif self.distribution == PriorDistribution.UNIFORM:
692                 if self.lower is None or self.upper is None:
693                     raise ValueError("uniform prior requires lower and upper")
694                 if self.lower >= self.upper:
695                     raise ValueError("uniform lower must be < upper")
696         return self

```

697 5.5 Loading and Validating YAML Files

698 To load and validate a target:

```

699 import yaml
700 from maple.core.calibration import SubmodelTarget
701
702 # Load YAML file
703 with open("psc_proliferation_PDAC_deriv001.yaml") as f:
704     data = yaml.safe_load(f)
705
706 # Parse and validate (raises ValidationError on failure)
707 target = SubmodelTarget(**data)
708
709 # Access validated fields
710 print(f"Target: {target.target_id}")
711 print(f"Parameters: {[p.name for p in target.calibration.parameters]}")
712 print(f"Model type: {target.calibration.model.type}")

```

713 If validation fails, Pydantic raises a `ValidationError` with detailed error messages:

```

714 pydantic_core._pydantic_core.ValidationError: 2 validation errors for SubmodelTarget
715 calibration.measurements.0.uses_inputs.0
716   Input 'typo_input_name' not found in inputs [...]
717 primary_data_source.doi
718   DOI '10.1234/fake' failed to resolve [...]

```

719 5.6 CalibrationTarget Model

720 The `CalibrationTarget` schema represents clinical and in vivo endpoints for full-model calibration.
 721 Where `SubmodelTarget` isolates a single mechanism, `CalibrationTarget` operates on the full model
 722 state.

```

723 class CalibrationTarget(BaseModel):
724     """
725     Calibration target for full-model Bayesian inference.
726
727     Represents a clinical or in vivo measurement that constrains
728     model behavior at the system level.
729     """
730     # Observable: how to compute the measurement from model species
731     observable: Observable = Field(
732         description="Maps full model species to measured quantity"
733     )
734
735     # Empirical data: literature-derived distribution
736     empirical_data: CalibrationTargetEstimates = Field(
737         description="Observed values derived from literature via Monte Carlo"
738     )
739
740     # Clinical/experimental context
741     experimental_context: ExperimentalContext = Field(
742         description="Species, system, indication, treatment, staging"
743     )
744     scenario: Optional[Scenario] = Field(
745         default=None,
746         description="Treatment scenario for intervention studies"
747     )
748
749     # Interpretation and provenance (shared with SubmodelTarget)
750     study_interpretation: str
751     key_assumptions: List[str]
752     key_study_limitations: Optional[List[str]] = None
753     primary_data_source: PrimaryDataSource
754     secondary_data_sources: Optional[List[SecondaryDataSource]] = None
755     source_relevance: Optional[SourceRelevanceAssessment] = None
756
757     # 30+ validators (see Section S3)

```

5.7 Observable

The **Observable** specifies how to compute the experimental measurement from full model species. Unlike **SubmodelTarget**'s forward model (which defines isolated dynamics), the observable operates on the solved full-model state.

```
class Observable(BaseModel):
    """
    Full-model observable for CalibrationTarget.

    The code field contains a Python function that computes the
    measured quantity from model species at each time point.
    """
    code: str = Field(
        description="Python: compute_observable(time, species_dict, "
        "constants, ureg) -> Quantity array"
    )
    units: str = Field(description="Units of the observable output")
    species: List[str] = Field(
        description="Model species accessed (e.g., ['V_T.CD8', 'V_T.C1'])"
    )
    constants: Optional[List[ObservableConstant]] = Field(
        default=None,
        description="Named constants with provenance"
    )
    inputs: Optional[List[SubmodelInput]] = Field(
        default=None,
        description="Literature values used in the observable"
    )
    support: SupportType = Field(
        description="Mathematical domain: positive, non_negative, "
        "unit_interval, real"
    )
    mapping_rationale: str = Field(
        description="How the literature measurement maps to model species"
    )
    # Denominator audit fields for density/fraction observables
    experimental_denominator: Optional[str] = None
    model_denominator_species: Optional[List[str]] = None
```

Each **ObservableConstant** requires a `biological_basis` explanation and must trace to either the curated reference database or a specific literature source.

5.8 CalibrationTargetEstimates

The empirical data block derives summary statistics from literature inputs via Monte Carlo simulation.

```
class CalibrationTargetEstimates(BaseModel):
```

```

801     """
802     Empirical data block: literature values -> distribution for likelihood.
803     """
804     median: List[float] = Field(
805         description="Point estimates (length 1 for scalar, "
806             "or matching index_values for vector)"
807     )
808     ci95: List[List[float]] = Field(
809         description="95% credible intervals [[lo, hi], ...]"
810     )
811     units: str
812     sample_size: Union[int, List[int]]
813     sample_size_rationale: str
814
815     # For vector-valued data (time courses, dose responses)
816     index_values: Optional[List[float]] = None
817     index_unit: Optional[str] = None
818     index_type: Optional[IndexType] = None
819
820     # Inputs and assumptions
821     inputs: List[EstimateInput] = Field(
822         description="Literature values with provenance"
823     )
824     assumptions: Optional[List[ModelingAssumption]] = None
825
826     # Monte Carlo code
827     distribution_code: str = Field(
828         description="Python: derive_distribution(inputs, ureg) -> "
829             "{ 'median': [...], 'ci95': [...], 'units': str }"
830     )

```

831 5.9 Scenario and Intervention

832 For studies involving treatment, the `Scenario` captures the clinical context.

```

833 class Intervention(BaseModel):
834     """A single treatment intervention."""
835     agent: str = Field(description="Drug or treatment name")
836     dose: Optional[str] = None
837     route: Optional[str] = None
838     schedule: Optional[str] = None
839
840 class Scenario(BaseModel):
841     """Treatment scenario for intervention studies."""
842     interventions: List[Intervention]
843     treatment_line: Optional[str] = None
844     prior_treatments: Optional[List[str]] = None
845     description: str = Field(

```

```

846         description="Narrative description of the clinical scenario"
847     )

```

848 6 SubmodelTarget Schema Details

849 This section provides detailed field descriptions and representative YAML examples for each layer
850 of the SubmodelTarget schema. These examples are referenced from the Methods section; the
851 complete schema is defined in Section 5.

852 6.1 Input Fields

853 Each input captures a single value extracted from literature with full provenance. The key fields
854 are:

- 855 • **name:** Unique identifier referenced elsewhere in the schema
- 856 • **value** and **units:** The extracted numeric value with units
- 857 • **uncertainty:** Standard deviation or 95% confidence interval, when reported
- 858 • **n:** Sample size, for propagating uncertainty through the likelihood
- 859 • **input_type:** Classification as `direct_measurement`, `proxy_measurement`, `inferred_estimate`,
860 or `assumed_value`
- 861 • **role:** How the input is used: `target` (likelihood term), `auxiliary` (supporting), `initial_condition`,
862 or `fixed_parameter`
- 863 • **source_ref:** Reference to a literature source with DOI
- 864 • **value_snippet:** Quoted text from the source containing the value

865 Code block 1 shows a representative input.

```

inputs:
- name: pdgfr_fold_mean
  value: 4.37
  units: dimensionless
  uncertainty:
    sd: 0.49
  n: 3
  input_type: direct_measurement
  role: target
  source_ref: Schneider2001
  source_location: Abstract
  value_snippet: "Cell proliferation (4.37 +/- 0.49-
                  and 2.96 +/- 0.39-fold of control)."
```

Figure 1: An input capturing a proliferation measurement with provenance.

6.2 Parameter and Prior Specification

Each parameter includes a prior distribution with documented rationale. The schema supports four distribution families: LogNormal (positive parameters with multiplicative uncertainty), Normal (unconstrained), Uniform (bounded with no preference), and HalfNormal (positive with mode at zero).

Code block 2 shows a parameter specification.

```
calibration:
  parameters:
    - name: k_apsc_prolif
      units: 1/day
      prior:
        distribution: lognormal
        mu: 0.0      # log(1.0)
        sigma: 1.0
        rationale: |
          Wide prior centered at 1/day. A 4-fold increase
          in 1 day implies  $k \sim \ln(4) \sim 1.4/\text{day}$ .
```

Figure 2: A parameter with lognormal prior and documented rationale.

6.3 Forward Model Specification

The `forward_model` field captures the dynamics relevant to the experiment. For time-course data, this is typically a simplified ODE; for steady-state or derived measurements, an algebraic model may suffice. State variables and the independent-variable block are nested inside `forward_model`.

Code block 3 shows a forward model using the exponential growth template.


```

calibration:
  forward_model:
    type: exponential_growth
    rate_constant: k_apsc_prolif
    independent_variable:
      name: time
      units: day
      span: [0.0, 1.0]
    state_variables:
      - name: aPSC_rel
        units: dimensionless
        initial_condition:
          value: 1.0
          rationale: Normalized to control baseline.
    data_rationale: |
      BrdU assay measures proliferation over ~24h with
      constant PDGF stimulus. Exponential growth fits.
    submodel_rationale: |
      At saturating PDGF, the full model reduces to
       $dN/dt = k_{apsc\_prolif} * N$ .

```

Figure 3: Forward model specification with state variable and time span.

6.4 Error Model Specification

The `error_model` field is a list of measurement entries, each specifying the statistical relationship between model predictions and one observed quantity. The `observation_code` derives the observed value and uncertainty from raw inputs. The `observable` defines the transformation from state variables to predicted measurement. The `likelihood` specifies the distribution family.

Code block 4 shows a complete error model entry.

```

error_model:
  - name: viability_48h
    units: dimensionless
    uses_inputs: [viability_mean, viability_sem]
    evaluation_points: [2.0] # days
    sample_size: 3
    observable:
      type: identity
      state_variables: [aPSC_norm]
    observation_code: |
      def derive_observation(inputs, sample_size):
        mean = inputs['viability_mean']
        sem = inputs['viability_sem']
        sd = sem * np.sqrt(sample_size) # SEM to SD
        return {'value': mean, 'sd': sd}
  likelihood:
    distribution: normal
    rationale: |
      Viability near 1 with small SD; normal adequate.

```

Figure 4: Error model specification linking forward model predictions to observed data.

7 Complete CalibrationTarget Example

This section presents a complete CalibrationTarget for baseline CD8+ T cell density in PDAC, demonstrating how full-model observables, reference database constants, and Monte Carlo distribution derivation work together.

7.1 Overview

This target constrains baseline intratumoral CD8+ T cell density using immunohistochemistry data from Jansen et al. (2021), who measured CD8 density across 599 pancreatic cancers using tissue microarrays. The observable computes CD8+ density from full model species (effector and exhausted CD8+ T cells) and reference database constants (cancer cell geometry). The distribution code combines two patient subgroups via a mixture of lognormals.

7.2 Observable

The observable maps full model species to the experimental measurement:

```

observable:
  code: |
    def compute_observable(time, species_dict, constants, ureg):
      import numpy as np
      cd8_eff = species_dict['V_T.CD8']
      cd8_exh = species_dict['V_T.CD8_exh']
      C1 = species_dict['V_T.C1']
      cross_section = constants['pdac_cancer_cell_cross_section']

```

```

903         cellularity = constants['pdac_cellularity_fraction']
904
905         total_cd8 = cd8_eff + cd8_exh
906         tumor_area = C1 * cross_section / cellularity
907         density = total_cd8 / tumor_area
908         return density * ureg('cell / millimeter**2')
909     units: cell / millimeter**2
910     species: [V_T.CD8, V_T.CD8_exh, V_T.C1]
911     constants:
912     - name: pdac_cancer_cell_cross_section
913       value: 0.000227
914       units: millimeter**2 / cell
915       biological_basis: |
916           Mean cross-sectional area of PDAC cancer cells from
917           histomorphometric measurements. Derived from mean cell
918           diameter of 17 micrometers.
919       source_type: reference_db
920       reference_db_name: pdac_cancer_cell_cross_section
921     - name: pdac_cellularity_fraction
922       value: 0.15
923       units: dimensionless
924       biological_basis: |
925           Fraction of tumor volume occupied by cancer cells in PDAC.
926           PDAC tumors are characteristically desmoplastic with low
927           cellularity (10-20%).
928       source_type: reference_db
929       reference_db_name: pdac_cellularity_fraction
930     support: positive
931     mapping_rationale: |
932         IHC measures CD8+ cells per mm^2 of tissue section. The model
933         tracks CD8+ T cells as counts in the tumor compartment (V_T).
934         To convert model cell counts to density, we divide by tumor
935         area estimated from cancer cell count, cross-section, and
936         cellularity fraction.
937     experimental_denominator: "mm^2 of tumor tissue section area"
938     model_denominator_species: [V_T.C1]

```

939 7.3 Empirical Data

940 The distribution code combines two patient subgroups (DOG1-negative and DOG1-positive) via
941 mixture-of-lognormals Monte Carlo:

```

942 empirical_data:
943     median: [138.8]
944     ci95: [[20.3, 965.2]]
945     units: cell / millimeter**2
946     sample_size: 444
947     sample_size_rationale: |

```

948 Total of 444 patients: 368 DOG1-negative + 76 DOG1-positive.

949 inputs:

```
950     - name: mean_cd8_stroma_neg
951       value: 227.7
952       units: cell / millimeter**2
953       source_ref: Jansen2021
954       source_location: "Table 4"
955       value_snippet: "CD8 ... 227.7 +/- 15.0"
956     - name: sem_cd8_stroma_neg
957       value: 15.0
958       units: cell / millimeter**2
959       source_ref: Jansen2021
960       source_location: "Table 4"
961       value_snippet: "CD8 ... 227.7 +/- 15.0"
962     - name: n_stroma_neg
963       value: 368
964       units: dimensionless
965       source_ref: Jansen2021
966       source_location: "Table 1"
967       value_snippet: "DOG1 negative 368 (61.4%)"
968     - name: mean_cd8_stroma_pos
969       value: 220.1
970       units: cell / millimeter**2
971       source_ref: Jansen2021
972       source_location: "Table 4"
973       value_snippet: "CD8 ... 220.1 +/- 33.0"
974     - name: sem_cd8_stroma_pos
975       value: 33.0
976       units: cell / millimeter**2
977       source_ref: Jansen2021
978       source_location: "Table 4"
979       value_snippet: "CD8 ... 220.1 +/- 33.0"
980     - name: n_stroma_pos
981       value: 76
982       units: dimensionless
983       source_ref: Jansen2021
984       source_location: "Table 1"
985       value_snippet: "DOG1 positive 76 (12.7%)"
```

986 distribution_code: |

```
987     def derive_distribution(inputs, ureg):
988         import numpy as np
989         rng = np.random.default_rng(42)
990         n_mc = 100000
991
992         # Subgroup 1: DOG1-negative
993         mean1 = inputs['mean_cd8_stroma_neg'].magnitude
994         sem1 = inputs['sem_cd8_stroma_neg'].magnitude
995         n1 = int(inputs['n_stroma_neg'].magnitude)
```

```

996         sd1 = sem1 * np.sqrt(n1)
997
998         # Subgroup 2: DOG1-positive
999         mean2 = inputs['mean_cd8_stroma_pos'].magnitude
1000         sem2 = inputs['sem_cd8_stroma_pos'].magnitude
1001         n2 = int(inputs['n_stroma_pos'].magnitude)
1002         sd2 = sem2 * np.sqrt(n2)
1003
1004         # Convert to lognormal parameters
1005         def to_lognormal(m, s):
1006             var = s**2
1007             mu = np.log(m**2 / np.sqrt(m**2 + var))
1008             sigma = np.sqrt(np.log(1 + var / m**2))
1009             return mu, sigma
1010
1011         mu1, sig1 = to_lognormal(mean1, sd1)
1012         mu2, sig2 = to_lognormal(mean2, sd2)
1013
1014         # Mixture sampling
1015         w1 = n1 / (n1 + n2)
1016         mask = rng.random(n_mc) < w1
1017         samples = np.where(
1018             mask,
1019             rng.lognormal(mu1, sig1, n_mc),
1020             rng.lognormal(mu2, sig2, n_mc)
1021         )
1022
1023         return {
1024             'median': [float(np.median(samples))],
1025             'ci95': [[float(np.percentile(samples, 2.5)),
1026                     float(np.percentile(samples, 97.5))]],
1027             'units': 'cell / millimeter**2'
1028         }

```

1029 7.4 Source and Context

```

1030 primary_data_source:
1031     doi: "10.7717/peerj.11397"
1032     title: "Expression of immune cell markers in the tumor
1033            microenvironment and association with survival"
1034     authors: [Jansen, van Doorn, et al]
1035     year: 2021
1036     source_tag: Jansen2021
1037
1038 experimental_context:
1039     species: human
1040     system: clinical_resection
1041     indication: PDAC

```

```

1042     treatment_status: treatment_naive
1043     stage: resectable
1044
1045     study_interpretation: |
1046         Jansen et al. measured CD8+ T cell density across 599 pancreatic
1047         cancers using tissue microarrays and digital image analysis. We
1048         combine two subgroups (DOG1-positive and DOG1-negative) as a
1049         mixture of lognormals to capture the full population distribution.
1050
1051     key_assumptions:
1052         - "CD8 IHC detects both effector and exhausted T cells equally"
1053         - "Reported +/- values are SEM (confirmed by sqrt(n) test)"
1054         - "DOG1 expression does not significantly affect CD8 density"
1055
1056     key_study_limitations:
1057         - "TMA cores (0.6mm) may not represent whole-slide heterogeneity"
1058         - "Mixed pathologic stages without neoadjuvant therapy reporting"
1059         - "2D histology section vs. 3D model compartment"

```

1060 8 SubmodelTarget Source Characteristics

1061 This section provides detailed breakdowns of source quality, species translation, and indication
1062 match for the [37 curated](#) SubmodelTarget extractions.

1063 8.1 Source Quality

1064 Source quality categorizes the type of experimental evidence. *Human clinical* denotes data from hu-
1065 man patient samples or clinical measurements. *Human in vitro* denotes experiments using human-
1066 derived cell lines or primary human cells in culture. *Animal in vivo* denotes measurements from
1067 animal models (e.g., mouse xenografts). *Animal in vitro* denotes experiments using animal-derived
1068 cells in culture (e.g., murine splenocyte assays, rat stellate cell cultures).

Table 2: Distribution of primary data source quality across SubmodelTargets.

Source Quality	Count	%
primary_human_clinical	17	46%
primary_animal_in_vivo	9	24%
primary_animal_in_vitro	7	19%
primary_human_in_vitro	3	8%
review_article	1	3%
Total	37	

1069 8.2 Species Translation

1070 Species translation describes the mapping from the experimental organism to the model target
1071 species (human). Targets labeled human→human required no cross-species translation. Mixed-

species entries (e.g., human and rat→human) indicate that the extraction combined data from multiple species, such as a human clinical measurement anchored by rat-derived kinetic parameters.

Table 3: Species translation from data source to model target. Human→human indicates no cross-species translation required.

Translation	Count	%
human→human	21	57%
mouse→human	13	35%
rat→human	3	8%
Total	37	

8.3 Indication Match

Indication match describes how closely the experimental disease context matches the PDAC model target. *Exact*: data from PDAC patients or PDAC-derived models. *Proxy*: data from a different indication used as a mechanistic proxy (e.g., other cancer types, chronic pancreatitis, fibrosis models with similar stromal biology). *Related*: data from the same organ or disease class but not a direct match.

Table 4: How closely the data source indication matches the model target (PDAC).

Indication Match	Count	%
Data from PDAC patients/models	17	46%
Related indication (e.g., other cancers, fibrosis)	16	43%
related	4	11%
Total	37	

9 CalibrationTarget Detailed Metrics

This section provides detailed metrics for the 54 CalibrationTargets applied to the PDAC QSP model.

9.1 Observable Categories

9.2 Observable Complexity

The most complex observables (19 species) are neoadjuvant immunotherapy targets computing immune subset percentages of all nucleated cells, requiring all modeled immune, stromal, and tumor populations in the denominator. The highest-constant observables (6 constants) require geometric and cellularity reference parameters for converting between model cell counts and histologic measurements. Across all targets, observables reference 25 unique model species spanning immune cells (CD8⁺ effector and exhausted, Treg, Th, NK), myeloid cells (M1/M2 macrophages, MDSC, cDC1/cDC2), stromal cells (myCAF, iCAF, apCAF, quiescent PSC), tumor cells, ECM, and microenvironment variables (VEGF, lactate, pO₂, glucose, HIF).

Table 5: CalibrationTarget observable categories.

Observable Category	Count
Immune/stromal fractions	29
Cell densities	9
Population ratios	8
Metabolic/TME markers	3
Fold-changes	2
Clinical endpoints	2

Table 6: Observable complexity across 54 CalibrationTargets.

Metric	Mean	Min	Max
Model species per observable	7.4	1	19
Named constants per observable	0.6	0	6
Empirical inputs per target	5.7	1	18

9.3 Extraction Model Attribution

Table 7: CalibrationTarget extraction model breakdown.

Extraction Model	Count	Percentage
Claude Opus 4	27	50%
GPT-5.1	18	33%
Manually curated	9	16%
Total	54	

Of the 45 AI-generated targets, 30 (55%) are tagged as human-verified, indicating that a domain expert reviewed the complete target after initial extraction. 15 targets (27%) include data digitized from figures, where per-patient data points were manually extracted and the CalibrationTarget structure was assembled by the LLM from those data points.

9.4 Primary Data Sources

The 54 targets drew from 23 unique primary sources (by DOI), with publication years spanning 2000–2026. Two sources contributed the majority of targets: a neoadjuvant immunotherapy trial (Li et al., Cancer Cell, 2022) provided data for all 18 GVAX/nivolumab targets, and a large-scale immunohistochemistry study (Liudahl et al., Cancer Discovery, 2021) provided data for 12 baseline targets covering immune cell densities, fractions, and ratios.

10 Inference Results

All 37 SubmodelTargets were translated to a joint Julia inference script using Turing.jl¹. The translator identified 19 unique parameters and generated corresponding priors and likelihood terms. The plurality of targets (48%) use closed-form algebraic forward models; the remainder use structured model types including steady-state, batch accumulation, and first-order decay models (Table 8).

Table 8: Distribution of forward model types used in calibration submodels.

Model Type	Count	%
Algebraic (closed-form)	18	49%
Steady-state ratio	5	14%
Batch accumulation	4	11%
Steady-state density	4	11%
Steady-state fraction	3	8%
First-order decay ODE	2	5%
Steady-state concentration	1	3%
Total	37	

The generated script is self-contained, requiring only DifferentialEquations.jl, Turing.jl, and Distributions.jl². Each target contributes a simulate function, observed data constants, and a likelihood term. In this application, most targets constrain individual parameters through algebraic or structured forward models (steady-state, batch accumulation, first-order decay), with 11 parameters shared across multiple targets. The framework supports arbitrarily complex scenarios: multiple derivations constraining the same parameter, single targets constraining multiple parameters, and forward models involving ODEs that cannot be analytically inverted.

10.1 Convergence Diagnostics

We ran 4 chains of 1000 samples each using the NUTS sampler. All parameters achieved $\hat{R} < 1.01$, with bulk-ESS > 2600 and tail-ESS > 1300 throughout (Table 9). Sampling took about 42 seconds on an M-series MacBook Air.

10.2 Posterior Distributions

Figure 5 shows posterior estimates for all 19 parameters. Table 10 reports medians and 90% credible intervals.

Most posteriors are narrower than their priors, indicating that the data constrained the parameters. The tumor growth rate `k_C1_growth` ($5.34\text{e-}03 \text{ day}^{-1}$, 90% CI: $2.22\text{e-}03\text{--}0.0128$) implies a doubling time of about 130 days, consistent with clinical PDAC volume-doubling-time measurements[?] [?]. The activated PSC death rate `k_apsc_death` (0.0388 day^{-1}) implies a half-life of about 10 days.

The IL-2-dependent division counts for CD4+ T cells (8.03) fall within the 7–8 generation range expected from CFSE proliferation tracking, where dye dilution limits counting. The CD8+ estimate (5.31) is somewhat lower, possibly reflecting differences in experimental conditions between the source studies.

11 Model-Aware Prompt Construction Details

For each target, the prompt injects mechanistic context extracted from the model structure. For target parameters, the system automatically retrieves: the parameter’s name, units, and biological description; the reactions where the parameter appears with their rate laws; the species involved in those reactions with descriptions; other parameters appearing in the same reactions; and the broader reaction network showing other reactions that involve the same species. For target observables,

Table 9: Convergence diagnostics for MCMC sampling. All $\hat{R} < 1.01$ indicates convergence.

Parameter	$\hat{R}$	ESS (bulk)	ESS (tail)
N_IL2_CD4	1.000	4598	2549
N_IL2_CD8	1.002	2846	1334
cd8_exclusion_fraction	1.001	2705	2325
k_C1_growth	1.001	4912	3195
k_CCL2_deg	1.001	5256	2763
k_CCL2_sec	1.001	5453	2998
k_CD8_death	1.001	2851	2889
k_M1_pol	1.001	3020	2396
k_M2_pol	1.001	2773	2542
k_MDSC_death	1.001	2679	2515
k_MDSC_rec	1.001	2657	2621
k_Mac_death	1.000	3154	2987
k_Mac_rec	1.001	3055	2861
k_Treg_death	1.000	3813	2318
k_apsc_death	1.000	4222	2208
k_psc_activation	1.000	4511	3033
n_CD8_clones	1.001	4657	2895
q_CD8_T_in	1.000	4492	2440
q_Treg_T_in	1.002	5077	2879

Table 10: Posterior parameter estimates from joint Bayesian inference. Credible intervals are 90% highest density intervals. [†]Auxiliary parameter introduced during extraction.

Parameter	Median	90% CI	Units
N_IL2_CD4	8.03	[5.73, 10.3]	dimensionless
N_IL2_CD8	5.31	[4.03, 5.99]	dimensionless
cd8_exclusion_fraction	0.912	[0.741, 0.982]	dimensionless
k_C1_growth	5.34e-03	[2.22e-03, 0.0128]	day ⁻¹
k_CCL2_deg	4.29	[2.77, 6.75]	hour ⁻¹
k_CCL2_sec	3.32e-10	[2.57e-10, 4.33e-10]	nmol·cell ⁻¹ ·day ⁻¹
k_CD8_death	2.23	[0.546, 7.55]	day ⁻¹
k_M1_pol	0.0392	[0.0298, 0.0519]	day ⁻¹
k_M2_pol	0.0776	[0.0545, 0.111]	day ⁻¹
k_MDSC_death	0.08	[0.0568, 0.114]	day ⁻¹
k_MDSC_rec	1.36e+07	[9.76e+06, 1.92e+07]	cell·mL ⁻¹ ·day ⁻¹
k_Mac_death	0.0219	[5.42e-03, 0.0812]	day ⁻¹
k_Mac_rec	8.92e+05	[2.11e+05, 4.08e+06]	cell·mL ⁻¹ ·day ⁻¹
k_Treg_death	0.2	[0.0664, 0.636]	day ⁻¹
k_apsc_death	0.0388	[0.0278, 0.0506]	day ⁻¹
k_psc_activation	0.5	[0.342, 0.737]	day ⁻¹
n_CD8_clones	10.3	[8.66, 12.2]	dimensionless
q_CD8_T_in	4.03e-04	[3.52e-04, 4.60e-04]	min ⁻¹ ·cm ⁻³
q_Treg_T_in	6.47e-04	[4.63e-04, 9.05e-04]	min ⁻¹ ·cm ⁻³

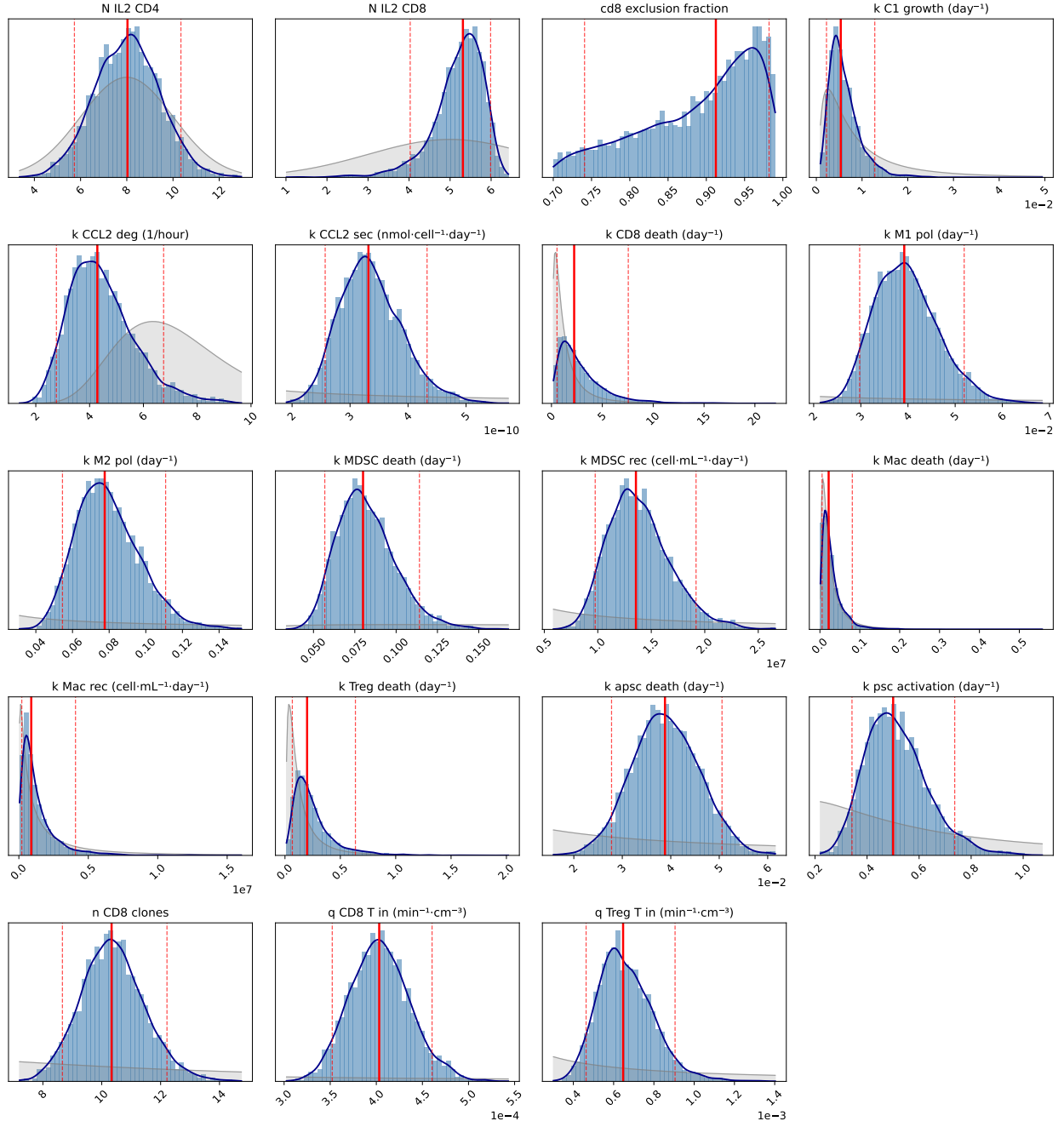


Figure 5: Posterior distributions from joint Bayesian inference. Gray shading shows the prior distribution; blue histogram and line show the posterior. Red lines indicate median (solid) and 90% credible interval bounds (dashed).

the system retrieves the observable’s name, units, and description, along with the species and compartments that define it. For example, when searching for data to calibrate a proliferation rate `k_apsc_prolif`, the prompt includes the reaction `aPSC -> 2*aPSC` with rate law `k_apsc_prolif * aPSC`, species descriptions explaining that `aPSC` represents activated pancreatic stellate cells, and related parameters like death rates that appear in the same reaction network.

This context guides the LLM’s search toward experiments that actually constrain the targets, rather than experiments that mention similar concepts but measure different quantities. For parameters, the broader reaction network helps the LLM identify whether additional parameters should be jointly calibrated. Extracting multiple parameters from the same study is intentional and preferred when the data supports it, as shared experimental conditions reduce inter-study variability.

To encourage literature diversity, source exclusion prevents the LLM from reusing papers across extractions. The prompt includes a list of primary sources already used for other targets, and a strict exclusion mode extends this to sources used by related parameters in the same reaction network.

Beyond mechanistic context, the prompt includes the complete schema structure with field descriptions, enabling the LLM to generate valid output without trial and error. The prompt also provides explicit guidance on common pitfalls: distinguishing SEM from SD (with conversion formulas), avoiding hardcoded numeric values in code blocks, and properly documenting source relevance for cross-species or cross-indication extractions. By showing validation requirements upfront (such as the prohibition on non-peer-reviewed sources and required uncertainty thresholds for proxy data), the prompt reduces the retry loop by preventing predictable errors.

12 Schema Implementation Details

Both schemas are implemented using Pydantic³, a Python library for declarative data validation. Each schema component is defined as a Pydantic model with typed fields and constraints. When YAML is parsed, Pydantic validates all constraints automatically: required fields must be present, values must match declared types, enumerations must use allowed values, and cross-field constraints must be satisfied.

The validation error messages are detailed and structured, reporting the field path, expected type, and actual value for each violation. This specificity is important for the LLM retry loop: when extraction fails validation, the error message provides actionable feedback. For example, a snippet validation error reports “Value-snippet mismatches (possible hallucination): input ‘cell_count_mean’ has value 4.37 but snippet contains ‘3.21’. Check that extracted values match the source text exactly.” This tells the LLM both what failed and how to fix it.

`SubmodelTarget` uses discriminated unions for its 15 forward model types: the `type` field determines which additional fields are required, ensuring type-specific validation without manual dispatch logic. `CalibrationTarget` uses a class hierarchy with shared base models for source documentation, relevance assessment, and experimental context.

13 Source Relevance Assessment Details

Each target includes structured metadata about its data sources. The `primary_data_source` block captures the main reference: DOI, title, authors, year, and a short `source_tag` used for cross-referencing elsewhere in the schema. Secondary sources that provide supporting context (e.g., reviews explaining biological mechanisms, or related studies used for uncertainty estimation) are

listed in `secondary_data_sources`, each with a `contribution` field explaining how that source informs the extraction.

The `indication_match` field classifies how well the source disease context matches the model: `exact` (same indication), `related` (same organ or disease class), `proxy` (different tissue used as mechanistic proxy), or `unrelated`. Cross-indication extractions require a justification explaining why the data is still informative.

Species translation is captured by `species_source` and `species_target` fields. When these differ (e.g., mouse data informing a human model), the extraction must document the assumed translation.

The `source_quality` field categorizes the evidence type: `primary_human_clinical`, `primary_human_in_vitro`, `primary_animal_in_vivo`, `primary_animal_in_vitro`, `review_article`, or `textbook`. By default, non-peer-reviewed sources trigger a validation warning; in practice, preprints (e.g., from bioRxiv) that have been publicly available for several months can provide valuable data, and the validator can be configured to accept them with documented justification.

For immune and stromal parameters, `tme_compatibility` assesses whether the tumor microenvironment in the source matches the target model (`high`, `moderate`, or `low`). A T cell trafficking rate measured in melanoma (a T cell-permissive tumor) may not translate directly to PDAC (an immunosuppressive tumor); low compatibility requires explicit documentation.

Finally, `estimated_translation_uncertainty_fold` quantifies the expected uncertainty from all translation factors combined. Cross-species extraction from a proxy indication with low TME compatibility might warrant 10-fold or greater uncertainty, which propagates to prior width during inference.

14 Collaboration Mode Details

The two collaboration modes observed in this study offer different trade-offs. In the batch pipeline mode, the LLM produces a structured draft that the modeler then curates interactively: selecting appropriate forward model types and prior distributions for `SubmodelTargets`, correcting observable code and species mappings for `CalibrationTargets`, and revising source relevance assessments for both. This mode is efficient for initial coverage (many files per run) but carries a high rejection rate (58% in a documented batch review) because the LLM lacks real-time modeler guidance on source selection, species mapping, and denominator definitions. In the interactive mode, the modeler and LLM work together from the start, with the modeler directing extraction decisions while the LLM handles text generation, code writing, and literature comprehension. This mode produces near-final quality but requires deeper per-file engagement.

In both modes, the schema serves as the collaboration interface. It defines what information must be captured (ensuring completeness), constrains how it is represented (enabling validation), and separates data extraction from modeling decisions (clarifying where different types of expertise are needed). The validators provide guardrails during both LLM extraction and human editing, catching errors regardless of their source.

15 Detailed Comparison to Existing Approaches

General-purpose biomedical extraction frameworks like LLM-IE⁴ provide building blocks for named entity recognition and relation extraction, but do not address the downstream requirements of QSP calibration: uncertainty quantification, forward model specification, and code generation for

inference. Interactive systems like SciDaSynth⁵ support structured data extraction with human-in-the-loop refinement, but focus on tabular output rather than the hierarchical schema needed for QSP model calibration. QSP-Copilot⁶ applies LLMs to QSP model development broadly, including literature extraction, but emphasizes model structure discovery (entity-interaction pairs) rather than quantitative parameter calibration with provenance.

The validation approach in MAPLE draws on established strategies for hallucination detection. Requiring extracted values to appear in quoted source text is analogous to the grounding strategies used in retrieval-augmented generation⁷, where LLM outputs are verified against retrieved documents. The schema-based validation enforces structural constraints similar to those in constrained decoding and structured output systems, where JSON schemas guide LLM generation toward valid output⁸. The key contribution is combining these strategies with domain-specific validators (DOI resolution, unit parsing, code execution) in a framework tailored to quantitative scientific extraction.

Recent work on knowledge graph construction from biomedical literature^{9,10} demonstrates that LLMs can extract structured relationships at scale. The Immune Cell Knowledge Graph (ICKG) used zero-shot prompting with Llama 3.1 to extract activation and inhibition relationships from over 24,000 PubMed abstracts. Such approaches prioritize coverage and can tolerate some extraction errors because downstream analyses aggregate across many relationships. Parameter calibration has different requirements: each extracted value directly influences model predictions, so individual errors matter more. MAPLE trades some throughput for higher per-target validation stringency.

The IQ Consortium has outlined best practices for AI/ML in quantitative modeling, including the use of NLP for knowledge extraction from biomedical literature¹¹. They emphasize explainability, human-in-the-loop oversight, and consideration of context of use. MAPLE aligns with these principles: the schema documents reasoning and provenance, validators catch errors before human review, and the generated code is transparent and modifiable. The growing interest in coupling QSP with machine learning^{12,13,14,15} suggests that hybrid approaches combining mechanistic modeling with data-driven methods will become increasingly important; automated literature extraction is one component of this broader integration. Recent work has also explored using LLMs directly as tools for comparing and synthesizing perspectives on QSP methodology¹⁶. While we evaluate MAPLE on a QSP model, the approach is not limited to pharmacology: any parameter-rich mechanistic model that draws calibration data from literature (e.g., systems biology signaling networks, physiologically-based models) could benefit from structured extraction with validation.

References

- [1] Ge H, Xu K, Ghahramani Z. Turing: A Language for Flexible Probabilistic Inference. In: Proceedings of the Twenty-First International Conference on Artificial Intelligence and Statistics. PMLR; 2018. p. 1682-90. Available from: <https://proceedings.mlr.press/v84/ge18b.html>.
- [2] Besançon M, Papamarkou T, Anthoff D, Arslan A, Byrne S, Lin D, et al. Distributions.jl: Definition and Modeling of Probability Distributions in the JuliaStats Ecosystem. Journal of Statistical Software. 2021;98(16):1-30. Available from: <https://www.jstatsoft.org/v098/i16>. doi:10.18637/jss.v098.i16.
- [3] Colvin S, Jolibois E, Ramezani H, Garcia Badaracco A, Dorsey T, Montague D, et al.. Pydantic Validation; 2025. Available from: <https://github.com/pydantic/pydantic>.

- [4] Hsu E, Roberts K. LLM-IE: A Python Package for Biomedical Generative Information Extraction with Large Language Models. *JAMIA Open*. 2025 Apr;8(2):ooaf012. doi:10.1093/jamiaopen/ooaf012.
- [5] Wang X, Huey SL, Sheng R, Mehta S, Wang F. SciDaSynth: Interactive Structured Data Extraction From Scientific Literature With Large Language Model. *Campbell Systematic Reviews*. 2025;21(4):e70073. doi:10.1002/cl2.70073.
- [6] Saini A, Farnoud A. QSP-Copilot: An AI-Augmented Platform for Accelerating Quantitative Systems Pharmacology Model Development. *CPT: Pharmacometrics & Systems Pharmacology*. 2025;14(11):1775-86. doi:10.1002/psp4.70127.
- [7] Li Y, Fu X, Verma G, Buitelaar P, Liu M. Mitigating Hallucination in Large Language Models (LLMs): An Application-Oriented Survey on RAG, Reasoning, and Agentic Systems. *arXiv*; 2025. doi:10.48550/arXiv.2510.24476.
- [8] Geng S, Cooper H, Moskal M, Jenkins S, Berman J, Ranchin N, et al. JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models. *arXiv preprint arXiv:250110868*. 2025 Feb. doi:10.48550/arXiv.2501.10868.
- [9] He S, Tan Y, Ye Q, M Gubin M, Dede M, Rafei H, et al. AI-powered Immune Cell Knowledge Graph (ICKG) with Granular Immune Contexts Enables Immune Program Interpretation. *npj Artificial Intelligence*. 2026 Jan;2(1):13. doi:10.1038/s44387-025-00060-4.
- [10] Gao T, Zhai X, Yang C, Lv L, Wang H. Joint Extraction of Entity and Relation Based on Fine-Tuning BERT for Long Biomedical Literatures. *Bioinformatics Advances*. 2024 Jan;4(1):vbae194. doi:10.1093/bioadv/vbae194.
- [11] Terranova N, Renard D, Shahin MH, Menon S, Cao Y, Hop CECA, et al. Artificial Intelligence for Quantitative Modeling in Drug Discovery and Development: An Innovation and Quality Consortium Perspective on Use Cases and Best Practices. *Clinical Pharmacology & Therapeutics*. 2024;115(4):658-72. doi:10.1002/cpt.3053.
- [12] Folguera-Blasco N, Boshier FAT, Uatay A, Pichardo-Almarza C, Lai M, Biasetti J, et al. Coupling Quantitative Systems Pharmacology Modelling to Machine Learning and Artificial Intelligence for Drug Development: Its pAIns and gAIns. *Frontiers in Systems Biology*. 2024 Jul;4. doi:10.3389/fsysb.2024.1380685.
- [13] Pawłowski T, Bokota G, Lazarou G, Kierzek AM, Sroka J. Emulation of Quantitative Systems Pharmacology Models to Accelerate Virtual Population Inference in Immuno-Oncology. *Methods (San Diego, Calif)*. 2024 Mar;223:118-26. doi:10.1016/j.ymeth.2023.12.006.
- [14] Goryanin I, Goryanin I, Demin O. Revolutionizing Drug Discovery: Integrating Artificial Intelligence with Quantitative Systems Pharmacology. *Drug Discovery Today*. 2025 Sep;30(9):104448. doi:10.1016/j.drudis.2025.104448.
- [15] Androulakis IP, Cucurull-Sanchez L, Kondic A, Mehta K, Pichardo C, Pryor M, et al. The Dawn of a New Era: Can Machine Learning and Large Language Models Reshape QSP Modeling? *Journal of Pharmacokinetics and Pharmacodynamics*. 2025 Jun;52(4):36. doi:10.1007/s10928-025-09984-5.

- 1304 [16] Androulakis IP, Cheng L, Cho CR, Zhang T. Leveraging Large Language Models to Compare
1305 Perspectives on Integrating QSP and AI/ML. *Journal of Pharmacokinetics and Pharmacody-*
1306 *namics*. 2025 May;52(3):29. doi:10.1007/s10928-025-09976-5.